Extra EXERCISE of Node Js

AN ISO 9001:2015 CERTIFIED INSTITUTE

COSMOS

The Real Training Centre

Extra Exercise of Node Js

1. **Command-Line Greeting (Simple Hello World)**

   Definition: A program that greets the user by name, which is passed as an argument through the command line.

   Example:

```
// greet.js
const name = process.argv[2];

if (!name) {
  console.log('Please provide your name!');
  process.exit(1);  // Exit the program if no name is
provided
}
console.log(`Hello, ${name}! Welcome to Node.js!`);
```

2. **Simple Command-Line Calculator**

   Definition: A basic calculator that accepts two numbers and an operator (+, -, *, /) as arguments and performs the calculation.

   Example:

```
// calculator.js
const args = process.argv.slice(2);
const num1 = parseFloat(args[0]);
const operator = args[1];
const num2 = parseFloat(args[2]);
if (!num1 || !num2 || !operator) {
  console.log('Usage: node calculator.js <num1> <operator>
<num2>');
  process.exit(1);
}
let result;
switch (operator) {
  case '+':
    result = num1 + num2;
    break;
  case '-':
    result = num1 - num2;
    break;
  case '*':
    result = num1 * num2;
    break;
  case '/':
```

```
      result = num1 / num2;
      break;
    default:
      console.log('Invalid operator. Use +, -, *, or /.');
      process.exit(1);
}
console.log(`Result: ${num1} ${operator} ${num2} =
${result}`);
```

3. Favorite Food Tracker

Definition: A program that takes a list of favorite foods and prints them out, each item on a new line.

Example:

```
// favoriteFood.js
const foods = process.argv.slice(2);
if (foods.length === 0) {
  console.log('Please provide at least one favorite food!');
  process.exit(1);
}
console.log('Your favorite foods are:');
foods.forEach((food, index) => {
  console.log(`${index + 1}. ${food}`);
});
```

4. Word Repeater (Print Word Multiple Times)

Definition: A program that takes a list of favorite foods and prints them out, each item on a new line.

Example:

```
// wordRepeater.js
const word = process.argv[2];
const repeatCount = parseInt(process.argv[3]);
if (!word || isNaN(repeatCount)) {
  console.log('Usage: node wordRepeater.js <word>
<repeatCount>');
  process.exit(1);
}
for (let i = 0; i < repeatCount; i++) {
  console.log(word);
}
```

5. Simple Countdown Timer

Definition: A program that counts down from a given number and displays the countdown on the console.

Example:

```
// countdown.js
const countdownStart = parseInt(process.argv[2]);

if (isNaN(countdownStart) || countdownStart <= 0) {
  console.log('Usage: node countdown.js <startNumber>');
  process.exit(1);
}
console.log('Countdown started:');
for (let i = countdownStart; i >= 0; i--) {
  console.log(i);
}
```

6. **Reading a File Asynchronously**

Definition: This example demonstrates how to use Node.js to read a file asynchronously. The fs.readFile() function is used to read the contents of a file without blocking the event loop.

Example:

```
// readFile.js
const fs = require('fs');
// Asynchronously reading a file
fs.readFile('example.txt', 'utf-8', (err, data) => {
  if (err) {
    console.error('Error reading the file:', err);
    return;
  }
  console.log('File content:', data);
});
```

7. **Writing to a File Asynchronously**

Definition: In this example, we write data to a file asynchronously using fs.writeFile(). This function creates a new file or overwrites an existing file with the given data.

Example:

```
// writeFile.js
const fs = require('fs');
// Asynchronously writing to a file
const content = 'This is some text I want to write to the
file.';
fs.writeFile('output.txt', content, (err) => {
  if (err) {
    console.error('Error writing to the file:', err);
    return;
  }
  console.log('File has been saved!');});});
```

8. Appending Data to a File

Definition: This example shows how to append data to an existing file using fs.appendFile(). If the file does not exist, it is created.

Example:

```
// appendFile.js
const fs = require('fs');
// Data to append to the file
const additionalContent = '\nThis is an additional line
appended to the file.';
fs.appendFile('output.txt', additionalContent, (err) => {
  if (err) {
    console.error('Error appending to the file:', err);
    return;
  }
  console.log('Data has been appended to the file!');
});
```

9. Checking if a File Exists

Definition: This example demonstrates how to check if a file exists using fs.existsSync(). It returns true if the file exists and false if it doesn't.

Example:

```
// checkFileExistence.js
const fs = require('fs');

// Check if the file exists
const filePath = 'example.txt';

if (fs.existsSync(filePath)) {
  console.log(`${filePath} exists.`);
} else {
  console.log(`${filePath} does not exist.`);
}
```

10. Renaming a File

Definition: This example shows how to rename a file using fs.rename(). It allows you to change the name of an existing file or move it to a different location.

Example:

```
// renameFile.js
const fs = require('fs');
// Renaming a file
const oldFileName = 'output.txt';
const newFileName = 'new_output.txt';
fs.rename(oldFileName, newFileName, (err) => {
  if (err) {
```

```
      console.error('Error renaming the file:', err);
      return;
    }
    console.log(`File renamed from ${oldFileName} to
${newFileName}`);
});
```

11. Simple Task Management App (CRUD)

Definition: A simple task management API where you can create, view, update, and delete tasks.

Example:

```
const express = require('express');
const app = express();
app.use(express.json());
let tasks = [];  // In-memory database
// GET all tasks
app.get('/tasks', (req, res) => {
  res.json(tasks);
});
// POST create a new task
app.post('/tasks', (req, res) => {
  const { title, description } = req.body;
  const newTask = { id: tasks.length + 1, title,
description, completed: false };
  tasks.push(newTask);
  res.status(201).json(newTask);
});
// PUT update a task
app.put('/tasks/:id', (req, res) => {
  const taskId = parseInt(req.params.id);
  const { title, description, completed } = req.body;
  const task = tasks.find(t => t.id === taskId);
  if (task) {
    task.title = title || task.title;
    task.description = description || task.description;
    task.completed = completed !== undefined ? completed :
task.completed;
    res.json(task);
  } else {
    res.status(404).json({ message: 'Task not found' });
  }
});
// DELETE a task
app.delete('/tasks/:id', (req, res) => {
  const taskId = parseInt(req.params.id);
```

```
  const index = tasks.findIndex(t => t.id === taskId);
  if (index !== -1) {
    tasks.splice(index, 1);
    res.json({ message: 'Task deleted' });
  } else {
    res.status(404).json({ message: 'Task not found' });
  }
});
app.listen(3000, () => console.log('Server running on
http://localhost:3000'));
```

12. User Profile Management API

Definition: An API to manage user profiles where you can add, update, view, and delete profiles.

Example:

```
const express = require('express');
const app = express();
app.use(express.json());
let profiles = [];
// GET all profiles
app.get('/profiles', (req, res) => {
  res.json(profiles);
});
// POST create a new profile
app.post('/profiles', (req, res) => {
  const { name, age, email } = req.body;
  const newProfile = { id: profiles.length + 1, name, age,
email };
  profiles.push(newProfile);
  res.status(201).json(newProfile);
});
// PUT update a profile
app.put('/profiles/:id', (req, res) => {
  const profileId = parseInt(req.params.id);
  const { name, age, email } = req.body;
  const profile = profiles.find(p => p.id === profileId);
  if (profile) {
    profile.name = name || profile.name;
    profile.age = age || profile.age;
    profile.email = email || profile.email;
    res.json(profile);
  } else {
    res.status(404).json({ message: 'Profile not found' });
  }
});
```

```
// DELETE a profile
app.delete('/profiles/:id', (req, res) => {
  const profileId = parseInt(req.params.id);
  const index = profiles.findIndex(p => p.id === profileId);
  if (index !== -1) {
    profiles.splice(index, 1);
    res.json({ message: 'Profile deleted' });
  } else {
    res.status(404).json({ message: 'Profile not found' });
  }
});
app.listen(3000, () => console.log('Server running on
http://localhost:3000'));
```

13. Blog Post CRUD API

Definition: A simple blog API to manage blog posts, where you can create, update, view, and delete posts.

Example:

```
const express = require('express');
const app = express();
app.use(express.json());
let posts = [];
// GET all blog posts
app.get('/posts', (req, res) => {
  res.json(posts);
});
// POST create a new blog post
app.post('/posts', (req, res) => {
  const { title, content, author } = req.body;
  const newPost = { id: posts.length + 1, title, content,
author, createdAt: new Date() };
  posts.push(newPost);
  res.status(201).json(newPost);
});
// PUT update a blog post
app.put('/posts/:id', (req, res) => {
  const postId = parseInt(req.params.id);
  const { title, content } = req.body;
  const post = posts.find(p => p.id === postId);
  if (post) {
    post.title = title || post.title;
    post.content = content || post.content;
    res.json(post);
  } else {
    res.status(404).json({ message: 'Post not found' });
```

```
  }
});
// DELETE a blog post
app.delete('/posts/:id', (req, res) => {
  const postId = parseInt(req.params.id);
  const index = posts.findIndex(p => p.id === postId);
  if (index !== -1) {
    posts.splice(index, 1);
    res.json({ message: 'Post deleted' });
  } else {
    res.status(404).json({ message: 'Post not found' });
  }
});
app.listen(3000, () => console.log('Server running on
http://localhost:3000'));
```

14. Simple Shopping Cart API

Definition: A shopping cart API where you can add, update, view, and remove items from the cart.

Example:

```
const express = require('express');
const app = express();
app.use(express.json());
let cart = [];
// GET the shopping cart
app.get('/cart', (req, res) => {
  res.json(cart);
});
// POST add an item to the cart
app.post('/cart', (req, res) => {
  const { item, quantity, price } = req.body;
  const newItem = { id: cart.length + 1, item, quantity,
price };
  cart.push(newItem);
  res.status(201).json(newItem);
});
// PUT update an item in the cart
app.put('/cart/:id', (req, res) => {
  const itemId = parseInt(req.params.id);
  const { quantity, price } = req.body;
  const cartItem = cart.find(c => c.id === itemId);
  if (cartItem) {
    cartItem.quantity = quantity || cartItem.quantity;
    cartItem.price = price || cartItem.price;
    res.json(cartItem);
```

```
    } else {
      res.status(404).json({ message: 'Item not found' });
    }
  });
  // DELETE an item from the cart
  app.delete('/cart/:id', (req, res) => {
    const itemId = parseInt(req.params.id);
    const index = cart.findIndex(c => c.id === itemId);
    if (index !== -1) {
      cart.splice(index, 1);
      res.json({ message: 'Item removed from cart' });
    } else {
      res.status(404).json({ message: 'Item not found' });
    }
  });
  app.listen(3000, () => console.log('Server running on
  http://localhost:3000'));
```

15. Movie Collection API

Definition: A simple API for managing a movie collection where you can add, update, view, and delete movies.

Example:

```
const express = require('express');
const app = express();
app.use(express.json());
let movies = [];
// GET all movies
app.get('/movies', (req, res) => {
  res.json(movies);
});
// POST add a movie
app.post('/movies', (req, res) => {
  const { title, genre, director } = req.body;
  const newMovie = { id: movies.length + 1, title, genre,
director };
  movies.push(newMovie);
  res.status(201).json(newMovie);
});
// PUT update a movie
app.put('/movies/:id', (req, res) => {
  const movieId = parseInt(req.params.id);
  const { title, genre, director } = req.body;
  const movie = movies.find(m => m.id === movieId);
  if (movie) {
    movie.title = title || movie.title;
```

```
    movie.genre = genre || movie.genre;
    movie.director = director || movie.director;
    res.json(movie);
  } else {
    res.status(404).json({ message: 'Movie not found' });
  }
});
// DELETE a movie
app.delete('/movies/:id', (req, res) => {
  const movieId = parseInt(req.params.id);
  const index = movies.findIndex(m => m.id === movieId);
  if (index !== -1) {
    movies.splice(index, 1);
    res.json({ message: 'Movie deleted' });
  } else {
    res.status(404).json({ message: 'Movie not found' });
  }
});
app.listen(3000, () => console.log('Server running on
http://localhost:3000'));
```

16. Request Logging Middleware

Definition: A middleware that logs every incoming request to the server. This can be helpful for debugging or tracking requests.

Example:

```
const express = require('express');
const app = express();
// Logging middleware
app.use((req, res, next) => {
  const currentDate = new Date();
  console.log(`[${currentDate}] ${req.method} request to
${req.url}`);
  next(); // Pass the control to the next middleware/route
});
app.get('/', (req, res) => {
  res.send('Hello, World!');
});
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

17. Authentication Middleware

Definition: A middleware that checks whether a request contains a valid authentication token before allowing access to a specific route.

Example:

```
const express = require('express');
const app = express();
// Sample authentication middleware
const authenticate = (req, res, next) => {
  const token = req.headers['authorization'];
  if (token && token === 'valid-token') {
    next(); // Proceed to the next middleware/route handler
  } else {
    res.status(401).json({ message: 'Unauthorized. Invalid
token.' });
  }
};
// Protected route
app.get('/dashboard', authenticate, (req, res) => {
  res.send('Welcome to the Dashboard!');
});
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

18. Custom Data Transformation Middleware

Definition: A middleware that processes incoming data and adds custom properties to the request object before it reaches the route handler.

Example:

```
const express = require('express');
const app = express();
// Custom data processing middleware
app.use((req, res, next) => {
  if (req.method === 'POST' && req.body) {
    req.body.processedData = `Processed: ${req.body.data}`;
  }
  next(); // Pass control to the next middleware or route
});
app.post('/process-data', express.json(), (req, res) => {
  res.json({ original: req.body.data, processed:
req.body.processedData });
});
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

19. Error Handling Middleware

Definition: A middleware specifically designed to handle errors. It catches any errors that occur during request processing and formats the response accordingly.

Example:

```
const express = require('express');
const app = express();
// Simulate an error in a route
app.get('/error', (req, res, next) => {
  const error = new Error('Something went wrong!');
  next(error); // Pass the error to the next middleware
});
// Error handling middleware
app.use((err, req, res, next) => {
  console.error(err.message);
  res.status(500).json({ message: 'Internal Server Error',
error: err.message });
});
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

20. Request Time Middleware

Definition: A middleware that adds the request's processing time to the response so the client knows how long the server took to handle the request.

Example:

```
const express = require('express');
const app = express();
// Middleware to track request time
app.use((req, res, next) => {
  const start = Date.now(); // Record the start time
  res.on('finish', () => {
    const duration = Date.now() - start; // Calculate the
time taken
    console.log(`Request to ${req.url} took ${duration}ms`);
  });
  next(); // Pass control to the next middleware or route
});
app.get('/', (req, res) => {
  res.send('Hello, World!');
});
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

21. MongoDB: Simple Todo List with User Authentication

Definition: This example shows how to create a simple Todo List application where users can register, log in, and add/update/delete tasks. This example also uses JWT (JSON Web Tokens) for user authentication.

Example:

```
const express = require('express');
const mongoose = require('mongoose');
const jwt = require('jsonwebtoken');
const bcrypt = require('bcrypt');
const app = express();
app.use(express.json());
// Connect to MongoDB
mongoose.connect('mongodb://localhost:27017/todolist', {
  useNewUrlParser: true,
  useUnifiedTopology: true,
});
// Mongoose Schemas and Models
const userSchema = new mongoose.Schema({
  username: String,
  password: String,
});
const User = mongoose.model('User', userSchema);
const taskSchema = new mongoose.Schema({
  title: String,
  completed: { type: Boolean, default: false },
  userId: { type: mongoose.Schema.Types.ObjectId, ref:
'User' },
});
const Task = mongoose.model('Task', taskSchema);
// User registration
app.post('/register', async (req, res) => {
  const { username, password } = req.body;
  const hashedPassword = await bcrypt.hash(password, 10);
  const newUser = new User({ username, password:
hashedPassword });
  await newUser.save();
  res.status(201).json({ message: 'User registered' });
});
// User login (returns JWT)
app.post('/login', async (req, res) => {
  const { username, password } = req.body;
  const user = await User.findOne({ username });
  if (user && await bcrypt.compare(password, user.password))
{
    const token = jwt.sign({ userId: user._id },
'secretKey');
    res.json({ token });
  } else {
    res.status(401).json({ message: 'Invalid credentials'
});
```

```
  }
});
// Middleware to verify JWT token
const authenticate = (req, res, next) => {
  const token = req.headers['authorization'];
  if (!token) return res.status(403).json({ message: 'No
token provided' });
  jwt.verify(token, 'secretKey', (err, decoded) => {
    if (err) return res.status(401).json({ message: 'Invalid
token' });
    req.userId = decoded.userId;
    next();
  });
};
// Create task
app.post('/tasks', authenticate, async (req, res) => {
  const { title } = req.body;
  const newTask = new Task({ title, userId: req.userId });
  await newTask.save();
  res.status(201).json(newTask);
});
// Get user's tasks
app.get('/tasks', authenticate, async (req, res) => {
  const tasks = await Task.find({ userId: req.userId });
  res.json(tasks);
});
// Update task
app.put('/tasks/:id', authenticate, async (req, res) => {
  const { title, completed } = req.body;
  const updatedTask = await Task.findByIdAndUpdate(
    req.params.id,
    { title, completed },
    { new: true }
  );
  res.json(updatedTask);
});
// Delete task
app.delete('/tasks/:id', authenticate, async (req, res) => {
  await Task.findByIdAndDelete(req.params.id);
  res.json({ message: 'Task deleted' });
});
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

22. MongoDB: Product Catalog with Aggregation

Definition: This example demonstrates how to create a Product Catalog application where you can filter products by price and category, and also perform advanced aggregation operations.

Example:

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();
app.use(express.json());
// Connect to MongoDB
mongoose.connect('mongodb://localhost:27017/productcatalog',
{
  useNewUrlParser: true,
  useUnifiedTopology: true,
});
// Mongoose Schema and Model
const productSchema = new mongoose.Schema({
  name: String,
  category: String,
  price: Number,
  stock: Number,
});
const Product = mongoose.model('Product', productSchema);
// Create product
app.post('/products', async (req, res) => {
  const { name, category, price, stock } = req.body;
  const newProduct = new Product({ name, category, price,
stock });
  await newProduct.save();
  res.status(201).json(newProduct);
});
// Get products by category and price range with aggregation
app.get('/products', async (req, res) => {
  const { category, minPrice, maxPrice } = req.query;
  const match = {};
  if (category) match.category = category;
  if (minPrice || maxPrice) match.price = {};
  if (minPrice) match.price.$gte = parseFloat(minPrice);
  if (maxPrice) match.price.$lte = parseFloat(maxPrice);
  const products = await Product.aggregate([
    { $match: match },
    { $sort: { price: 1 } },
    { $project: { name: 1, category: 1, price: 1 } },
  ]);
  res.json(products);
```

```
});
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

23. MongoDB: Real-Time Chat Application

Definition: A simple real-time chat application where users can send messages to each other. MongoDB is used to store chat history, and WebSockets are used to allow real-time messaging**.**

Example:

```
const express = require('express');
const mongoose = require('mongoose');
const http = require('http');
const socketIo = require('socket.io');
const app = express();
const server = http.createServer(app);
const io = socketIo(server);
app.use(express.json());
// Connect to MongoDB
mongoose.connect('mongodb://localhost:27017/chatapp', {
  useNewUrlParser: true,
  useUnifiedTopology: true,
});
// Mongoose Schema and Model
const messageSchema = new mongoose.Schema({
  user: String,
  message: String,
  createdAt: { type: Date, default: Date.now },
});
const Message = mongoose.model('Message', messageSchema);
// Get chat history
app.get('/messages', async (req, res) => {
  const messages = await Message.find().sort({ createdAt: 1
});
  res.json(messages);
});
// WebSocket for real-time communication
io.on('connection', (socket) => {
  console.log('A user connected');
  // Listen for incoming messages
  socket.on('sendMessage', async (data) => {
    const { user, message } = data;
    const newMessage = new Message({ user, message });
    await newMessage.save();
    io.emit('receiveMessage', newMessage);
```

```
  });
  socket.on('disconnect', () => {
    console.log('User disconnected');
  });
});
server.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

24. MongoDB: Blogging Platform with Comments

Definition: A blogging platform where users can create blog posts and comment on them. This example demonstrates relationships between MongoDB documents by embedding comments within the blog posts.

Example:

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();
app.use(express.json());
// Connect to MongoDB
mongoose.connect('mongodb://localhost:27017/blogging', {
  useNewUrlParser: true,
  useUnifiedTopology: true,
});
// Mongoose Schemas and Models
const commentSchema = new mongoose.Schema({
  user: String,
  text: String,
  createdAt: { type: Date, default: Date.now },
});
const postSchema = new mongoose.Schema({
  title: String,
  content: String,
  comments: [commentSchema],
});
const Post = mongoose.model('Post', postSchema);
// Create blog post
app.post('/posts', async (req, res) => {
  const { title, content } = req.body;
  const newPost = new Post({ title, content });
  await newPost.save();
  res.status(201).json(newPost);
});
// Add comment to a post
app.post('/posts/:id/comments', async (req, res) => {
  const { id } = req.params;
```

```
  const { user, text } = req.body;
  const post = await Post.findById(id);
  post.comments.push({ user, text });
  await post.save();
  res.status(201).json(post);
});
// Get blog post with comments
app.get('/posts/:id', async (req, res) => {
  const post = await Post.findById(req.params.id);
  res.json(post);
});
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

25. MongoDB: Task Management with Subtasks

Definition: This example showcases a Task Management application where tasks can have multiple subtasks, and MongoDB is used to model the hierarchy.

Example:

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();
app.use(express.json());
// Connect to MongoDB
mongoose.connect('mongodb://localhost:27017/taskmanagement',
{
  useNewUrlParser: true,
  useUnifiedTopology: true,
});
// Mongoose Schema and Model
const subtaskSchema = new mongoose.Schema({
  title: String,
  completed: { type: Boolean, default: false },
});
const taskSchema = new mongoose.Schema({
  title: String,
  description: String,
  subtasks: [subtaskSchema],
});
const Task = mongoose.model('Task', taskSchema);
// Create a task with subtasks
app.post('/tasks', async (req, res) => {
  const { title, description, subtasks } = req.body;
```

```
  const newTask = new Task({ title, description, subtasks
});
  await newTask.save();
  res.status(201).json(newTask);
});
// Add a subtask to an existing task
app.post('/tasks/:id/subtasks', async (req, res) => {
  const { id } = req.params;
  const { title } = req.body;
  const task = await Task.findById(id);
  task.subtasks.push({ title });
  await task.save();
  res.status(201).json(task);
});
// Get a task with subtasks
app.get('/tasks/:id', async (req, res) => {
  const task = await Task.findById(req.params.id);
  res.json(task);
});
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

26. Mongoose: User Profile Management with Validation and Timestamps

Definition: This example shows how to build a user profile management system where users can create, update, and delete their profiles. We'll use Mongoose to add validation, default values, and timestamps for profile data.

Example:

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();
app.use(express.json());
// Connect to MongoDB
mongoose.connect('mongodb://localhost:27017/userprofiles', {
  useNewUrlParser: true,
  useUnifiedTopology: true,
});
// Mongoose Schema and Model
const userProfileSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
      minlength: [3, 'Name should be at least 3 characters
long'],
```

```
    },
    email: {
      type: String,
      required: true,
      unique: true,
      match: [/\S+@\S+\.\S+/, 'Please provide a valid
email'],
    },
    age: {
      type: Number,
      min: [18, 'Age should be at least 18'],
      max: [120, 'Age should be at most 120'],
    },
    bio: {
      type: String,
      default: 'This user has no bio yet.',
    },
  },
  { timestamps: true }
);
const UserProfile = mongoose.model('UserProfile',
userProfileSchema);
// Create user profile
app.post('/profile', async (req, res) => {
  const { name, email, age, bio } = req.body;
  try {
    const newProfile = new UserProfile({ name, email, age,
bio });
    await newProfile.save();
    res.status(201).json(newProfile);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});
// Get user profile
app.get('/profile/:id', async (req, res) => {
  try {
    const profile = await
UserProfile.findById(req.params.id);
    if (!profile) {
      return res.status(404).json({ message: 'Profile not
found' });
    }
    res.json(profile);
  } catch (err) {
```

```
      res.status(400).json({ error: err.message });
    }
  });
  // Update user profile
  app.put('/profile/:id', async (req, res) => {
    try {
      const updatedProfile = await
  UserProfile.findByIdAndUpdate(req.params.id, req.body, {
  new: true });
      res.json(updatedProfile);
    } catch (err) {
      res.status(400).json({ error: err.message });
    }
  });
  // Delete user profile
  app.delete('/profile/:id', async (req, res) => {
    try {
      await UserProfile.findByIdAndDelete(req.params.id);
      res.json({ message: 'Profile deleted successfully' });
    } catch (err) {
      res.status(400).json({ error: err.message });
    }
  });
  app.listen(3000, () => {
    console.log('Server running on http://localhost:3000');
  });
```

27. Mongoose: Blog Post with Embedded Comments

Definition: This example demonstrates how to build a blogging platform where each blog post can have multiple comments. Comments are embedded in the Post document using subdocuments in Mongoose.

Example:

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();
app.use(express.json());
// Connect to MongoDB
mongoose.connect('mongodb://localhost:27017/blogplatform', {
  useNewUrlParser: true,
  useUnifiedTopology: true,
});
// Mongoose Schema and Model
const commentSchema = new mongoose.Schema({
  user: String,
  text: String,
```

```
  createdAt: { type: Date, default: Date.now },
});
const postSchema = new mongoose.Schema(
  {
    title: String,
    content: String,
    comments: [commentSchema],
  },
  { timestamps: true }
);
const Post = mongoose.model('Post', postSchema);
// Create a new blog post
app.post('/posts', async (req, res) => {
  const { title, content } = req.body;
  const newPost = new Post({ title, content });
  await newPost.save();
  res.status(201).json(newPost);
});
// Add a comment to a blog post
app.post('/posts/:id/comments', async (req, res) => {
  const { user, text } = req.body;
  const post = await Post.findById(req.params.id);
  if (!post) return res.status(404).json({ message: 'Post
not found' });
  post.comments.push({ user, text });
  await post.save();
  res.status(201).json(post);
});
// Get a blog post with comments
app.get('/posts/:id', async (req, res) => {
  const post = await Post.findById(req.params.id);
  if (!post) return res.status(404).json({ message: 'Post
not found' });
  res.json(post);
});
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

28. Mongoose: Task Management with Status and Deadline

Definition: This example demonstrates a Task Management System where each task has a status (e.g., "To-Do", "In Progress", "Completed") and a deadline. Mongoose's enum and default values are used for data integrity.

Example:

```
const express = require('express');
```

```
const mongoose = require('mongoose');
const app = express();
app.use(express.json());
// Connect to MongoDB
mongoose.connect('mongodb://localhost:27017/taskmanagement',
{
  useNewUrlParser: true,
  useUnifiedTopology: true,
});
// Mongoose Schema and Model
const taskSchema = new mongoose.Schema({
  title: {
    type: String,
    required: true,
  },
  description: String,
  status: {
    type: String,
    enum: ['To-Do', 'In Progress', 'Completed'],
    default: 'To-Do',
  },
  deadline: Date,
});
const Task = mongoose.model('Task', taskSchema);
// Create a task
app.post('/tasks', async (req, res) => {
  const { title, description, deadline } = req.body;
  const newTask = new Task({ title, description, deadline
});
  await newTask.save();
  res.status(201).json(newTask);
});
// Get all tasks with status "In Progress"
app.get('/tasks/inprogress', async (req, res) => {
  const tasks = await Task.find({ status: 'In Progress' });
  res.json(tasks);
});
// Update task status
app.put('/tasks/:id/status', async (req, res) => {
  const { status } = req.body;
  const updatedTask = await Task.findByIdAndUpdate(
    req.params.id,
    { status },
    { new: true }
  );
```

```
    res.json(updatedTask);
});
app.listen(3000, () => {
    console.log('Server running on http://localhost:3000');
});
```

29. Mongoose: Product Catalog with Price Range Filtering

Definition: This example demonstrates how to build a Product Catalog API that allows users to filter products by price range using Mongoose's query methods.

Example:

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();
app.use(express.json());
// Connect to MongoDB
mongoose.connect('mongodb://localhost:27017/productcatalog',
{
    useNewUrlParser: true,
    useUnifiedTopology: true,
});
// Mongoose Schema and Model
const productSchema = new mongoose.Schema({
    name: String,
    description: String,
    price: { type: Number, min: 0 },
    stock: Number,
});
const Product = mongoose.model('Product', productSchema);
// Create a product
app.post('/products', async (req, res) => {
    const { name, description, price, stock } = req.body;
    const newProduct = new Product({ name, description, price,
stock });
    await newProduct.save();
    res.status(201).json(newProduct);
});
// Get products within a price range
app.get('/products', async (req, res) => {
    const { minPrice, maxPrice } = req.query;
    const query = {};
    if (minPrice) query.price = { $gte: minPrice };
    if (maxPrice) query.price = { ...query.price, $lte:
maxPrice };
    const products = await Product.find(query);
    res.json(products);
```

```
});
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

30. Mongoose: User Authentication with Hashed Passwords

Definition: This example demonstrates how to handle user authentication where passwords are securely stored using bcrypt for hashing and validation.

Example:

```
const express = require('express');
const mongoose = require('mongoose');
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');
const app = express();
app.use(express.json());
// Connect to MongoDB
mongoose.connect('mongodb://localhost:27017/userauth', {
  useNewUrlParser: true,
  useUnifiedTopology: true,
});
// Mongoose Schema and Model
const userSchema = new mongoose.Schema({
  username: { type: String, required: true, unique: true },
  password: { type: String, required: true },
});
const User = mongoose.model('User', userSchema);
// Register a user
app.post('/register', async (req, res) => {
  const { username, password } = req.body;
  const hashedPassword = await bcrypt.hash(password, 10);
  const newUser = new User({ username, password:
hashedPassword });
  await newUser.save();
  res.status(201).json({ message: 'User registered
successfully' });
});
// Login a user
app.post('/login', async (req, res) => {
  const { username, password } = req.body;
  const user = await User.findOne({ username });
  if (!user) return res.status(404).json({ message: 'User
not found' });
  const isPasswordValid = await bcrypt.compare(password,
user.password);
```

```
  if (!isPasswordValid) return res.status(401).json({
message: 'Invalid password' });
  const token = jwt.sign({ userId: user._id }, 'secretkey');
  res.json({ token });
});
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

31. MySQL: User Authentication System

Definition: This example demonstrates how to implement a user authentication system using MySQL in Node.js. The system includes registering users, hashing passwords with bcrypt, and allowing users to log in using JWT tokens.

Example:

```
const express = require('express');
const mysql = require('mysql');
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');
const app = express();
app.use(express.json());
// MySQL connection
const db = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: 'password',
  database: 'userauth'
});
db.connect(err => {
  if (err) throw err;
  console.log('Connected to MySQL');
});
// Register User
app.post('/register', (req, res) => {
  const { username, password } = req.body;
  bcrypt.hash(password, 10, (err, hashedPassword) => {
    if (err) return res.status(500).json({ error:
err.message });
    const sql = 'INSERT INTO users (username, password)
VALUES (?, ?)';
    db.query(sql, [username, hashedPassword], (err, result)
=> {
      if (err) return res.status(500).json({ error:
err.message });
      res.status(201).json({ message: 'User registered
successfully' });
```

```
    });
  });
});
// Login User
app.post('/login', (req, res) => {
  const { username, password } = req.body;
  const sql = 'SELECT * FROM users WHERE username = ?';
  db.query(sql, [username], (err, results) => {
    if (err) return res.status(500).json({ error:
err.message });
    if (results.length === 0) return res.status(404).json({
message: 'User not found' });

    bcrypt.compare(password, results[0].password, (err,
isMatch) => {
      if (err) return res.status(500).json({ error:
err.message });
      if (!isMatch) return res.status(401).json({ message:
'Invalid password' });

      const token = jwt.sign({ userId: results[0].id },
'secretkey');
      res.json({ token });
    });
  });
});
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

32. MySQL: Product Management System

Definition: A simple product management system that allows users to add, view, update, and delete products in the database using MySQL.

Example:

```
const express = require('express');
const mysql = require('mysql');
const app = express();
app.use(express.json());
// MySQL connection
const db = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: 'password',
  database: 'productsdb'
});
```

```
db.connect(err => {
  if (err) throw err;
  console.log('Connected to MySQL');
});
// Create product
app.post('/product', (req, res) => {
  const { name, price, stock } = req.body;
  const sql = 'INSERT INTO products (name, price, stock)
VALUES (?, ?, ?)';
  db.query(sql, [name, price, stock], (err, result) => {
    if (err) return res.status(500).json({ error:
err.message });
    res.status(201).json({ message: 'Product created
successfully' });
  });
});
// Get all products
app.get('/products', (req, res) => {
  const sql = 'SELECT * FROM products';
  db.query(sql, (err, results) => {
    if (err) return res.status(500).json({ error:
err.message });
    res.json(results);
  });
});
// Update product
app.put('/product/:id', (req, res) => {
  const { name, price, stock } = req.body;
  const sql = 'UPDATE products SET name = ?, price = ?,
stock = ? WHERE id = ?';
  db.query(sql, [name, price, stock, req.params.id], (err,
result) => {
    if (err) return res.status(500).json({ error:
err.message });
    res.json({ message: 'Product updated successfully' });
  });
});
// Delete product
app.delete('/product/:id', (req, res) => {
  const sql = 'DELETE FROM products WHERE id = ?';
  db.query(sql, [req.params.id], (err, result) => {
    if (err) return res.status(500).json({ error:
err.message });
    res.json({ message: 'Product deleted successfully' });
```

```
  });
});
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

33. MySQL: Relational Data with Joins (Orders and Customers)

Definition: This example demonstrates how to use JOINs to query related data from two tables: Customers and Orders.

Example:

```
const express = require('express');
const mysql = require('mysql');
const app = express();
// MySQL connection
const db = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: 'password',
  database: 'ordersdb'
});
db.connect(err => {
  if (err) throw err;
  console.log('Connected to MySQL');
});
// Get all orders with customer details
app.get('/orders', (req, res) => {
  const sql = `
    SELECT customers.name, customers.email,
orders.order_date, orders.total
    FROM orders
    INNER JOIN customers ON orders.customer_id =
customers.id
  `;
  db.query(sql, (err, results) => {
    if (err) return res.status(500).json({ error:
err.message });
    res.json(results);
  });
});
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

34. MySQL: Data Filtering with WHERE Clause

Definition: This example demonstrates how to use the WHERE clause to filter data based on specific conditions, such as fetching products based on price or stock.

Example:

```
const express = require('express');
const mysql = require('mysql');
const app = express();
// MySQL connection
const db = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: 'password',
  database: 'productsdb'
});
db.connect(err => {
  if (err) throw err;
  console.log('Connected to MySQL');
});
// Get products with price greater than a given value
app.get('/products/expensive', (req, res) => {
  const { price } = req.query;
  const sql = 'SELECT * FROM products WHERE price > ?';
  db.query(sql, [price], (err, results) => {
    if (err) return res.status(500).json({ error:
err.message });
    res.json(results);
  });
});
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

35. MySQL: Pagination with LIMIT and OFFSET

Definition: This example demonstrates how to implement pagination to retrieve a subset of records, helping to avoid overwhelming the client with too much data at once.

Example:

```
const express = require('express');
const mysql = require('mysql');
const app = express();
// MySQL connection
const db = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: 'password',
  database: 'productsdb'
```

```
});
db.connect(err => {
  if (err) throw err;
  console.log('Connected to MySQL');
});
// Get products with pagination
app.get('/products', (req, res) => {
  const { page = 1, limit = 10 } = req.query;
  const offset = (page - 1) * limit;
  const sql = 'SELECT * FROM products LIMIT ? OFFSET ?';
  db.query(sql, [parseInt(limit), offset], (err, results) =>
{
    if (err) return res.status(500).json({ error:
err.message });
    res.json(results);
  });
});
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

36. Multer: Uploading a Single Image with Resize (Using Sharp)

Definition: This example demonstrates how to upload a single image file, save it to the server, and resize it using Sharp before saving it.

Example:

```
const express = require('express');
const multer = require('multer');
const sharp = require('sharp');
const path = require('path');
const fs = require('fs');
const app = express();
// Set up Multer storage configuration
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    const uploadPath = './uploads/';
    if (!fs.existsSync(uploadPath)) {
      fs.mkdirSync(uploadPath);
    }
    cb(null, uploadPath);
  },
  filename: (req, file, cb) => {
    cb(null, Date.now() + path.extname(file.originalname));
  }
});
```

```
const upload = multer({ storage: storage });
// Single image upload route
app.post('/upload', upload.single('image'), (req, res) => {
  const filePath = path.join(__dirname, 'uploads',
req.file.filename);
  // Resize the uploaded image
  sharp(filePath)
    .resize(300, 300)
    .toFile(path.join(__dirname, 'uploads', 'resized-' +
req.file.filename), (err, info) => {
      if (err) {
        return res.status(500).json({ error: err.message });
      }
      res.json({
        message: 'Image uploaded and resized successfully!',
        file: req.file,
        info: info
      });
    });
});
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

37. Multer: Multiple File Upload with Limitations

Definition: This example demonstrates how to upload multiple files with a size limit using Multer. The uploaded files are stored in a specific folder.

Example:

```
const express = require('express');
const multer = require('multer');
const app = express();
// Set up Multer storage configuration
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, './uploads/');
  },
  filename: (req, file, cb) => {
    cb(null, Date.now() + '-' + file.originalname);
  }
});
// Set up file size limit (5 MB max per file)
const upload = multer({
  storage: storage,
  limits: { fileSize: 5 * 1024 * 1024 }, // 5MB limit
}).array('files', 5); // Accepts up to 5 files
```

```
// Multiple files upload route
app.post('/upload', (req, res) => {
  upload(req, res, (err) => {
    if (err instanceof multer.MulterError) {
      return res.status(500).json({ error: `Multer error:
${err.message}` });
    } else if (err) {
      return res.status(500).json({ error: `Unknown error:
${err.message}` });
    }
    res.json({
      message: 'Files uploaded successfully!',
      files: req.files
    });
  });
});
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

38. Multer: Upload Files with Custom File Types (Accepting Only Images)

Definition: This example demonstrates how to restrict uploads to image files only using Multer's file filter.

Example:

```
const express = require('express');
const multer = require('multer');
const path = require('path');
const app = express();
// Set up Multer storage configuration
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, './uploads/');
  },
  filename: (req, file, cb) => {
    cb(null, Date.now() + path.extname(file.originalname));
  }
});
// File filter to allow only image files
const fileFilter = (req, file, cb) => {
  const fileTypes = /jpeg|jpg|png|gif/;
  const extname =
fileTypes.test(path.extname(file.originalname).toLowerCase()
);
  const mimetype = fileTypes.test(file.mimetype);
```

```
    if (extname && mimetype) {
      return cb(null, true);
    } else {
      cb(new Error('Only image files are allowed!'), false);
    }
  };
  // Set up Multer upload with filter
  const upload = multer({
    storage: storage,
    fileFilter: fileFilter
  }).single('image');

  // Upload route for single image file
  app.post('/upload', (req, res) => {
    upload(req, res, (err) => {
      if (err) {
        return res.status(400).json({ error: err.message });
      }
      res.json({
        message: 'Image uploaded successfully!',
        file: req.file
      });
    });
  });
  app.listen(3000, () => {
    console.log('Server running on http://localhost:3000');
  });
```

39. Multer: Streaming Large Files to a Remote Server (S3)

Definition: This example demonstrates how to upload large files to a remote server (like Amazon S3) using Multer in combination with the AWS SDK. This avoids storing files on the local server.

Example:

```
const express = require('express');
const multer = require('multer');
const AWS = require('aws-sdk');
const multerS3 = require('multer-s3');
const app = express();
// Set up AWS S3 client
const s3 = new AWS.S3({
  accessKeyId: 'your-access-key-id',
  secretAccessKey: 'your-secret-access-key',
  region: 'us-east-1'
});
```

```
// Set up Multer to upload directly to S3
const upload = multer({
  storage: multerS3({
    s3: s3,
    bucket: 'your-bucket-name',
    acl: 'public-read',
    key: function (req, file, cb) {
      cb(null, Date.now().toString() + file.originalname);
    }
  })
}).single('file');
// Route for uploading a file to S3
app.post('/upload', (req, res) => {
  upload(req, res, (err) => {
    if (err) {
      return res.status(500).json({ error: 'Error uploading
to S3: ' + err.message });
    }
    res.json({
      message: 'File uploaded successfully to S3!',
      file: req.file
    });
  });
});
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

40. Multer: Uploading Files with Dynamic Naming Based on Metadata

Definition: This example shows how to dynamically generate file names based on metadata from the request, such as a user's ID or timestamp, before saving it.

Example:

```
const express = require('express');
const multer = require('multer');
const path = require('path');
const app = express();
// Set up Multer storage with dynamic filenames
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, './uploads/');
  },
  filename: (req, file, cb) => {
    const userId = req.body.userId || 'unknown-user';
    const timestamp = Date.now();
```

```
    cb(null, `${userId}-
${timestamp}${path.extname(file.originalname)}`);
  }
});
const upload = multer({ storage: storage }).single('file');
// Route for uploading files with dynamic filenames
app.post('/upload', (req, res) => {
  upload(req, res, (err) => {
    if (err) {
      return res.status(500).json({ error: err.message });
    }
    res.json({
      message: 'File uploaded successfully with dynamic
naming!',
      file: req.file
    });
  });
});
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```